

Gypsum Belt-Filter Cake Recognition

Automated valid / invalid detection from a plant camera feed

AI Expert — Final Project

Yaron Conroy

2026-09-23

Code: github.com/yaronconroy-del/picture-recognition-gypsum (private — access arranged on request)

Problem Statement

- A fixed camera watches a gypsum belt filter on a phosphoric-acid production line.
- Filtration quality is currently checked by manual visual inspection — intermittent, and inconsistent between operators/shifts.
- Goal: an image classifier that looks at a single camera frame and tells whether the cake is VALID (properly dewatered) or INVALID (wet gypsum), automatically and continuously.
- This is a proxy for a real yield-loss and equipment-damage problem, not a cosmetic visual check (see next slides).

Background: the Process

- Wet-process phosphoric acid production:
 - Phosphate rock + sulfuric acid → phosphoric acid + gypsum (by-product)
- Gypsum still carries residual phosphoric acid.
- It's washed with water, then dewatered on the belt filter this project's camera watches — to recover as much phosphoric acid as possible before the gypsum is discarded.
- “Invalid” = wet gypsum: the cake didn't dewater properly.

Rock + H₂SO₄



Phosphoric acid + Gypsum (wet)

↓ wash with water

↓ belt filter (dewater)

Dry cake → recovered acid | Wet cake → lost acid

Background: the Cost of Bad Filtration

- Wet gypsum causes two separate costs:
 - Yield loss — unrecovered phosphoric acid, lost with the discarded gypsum: ~\$2,000/hour per 1% P₂O₅ remaining
 - Equipment damage — wet gypsum can damage downstream equipment, adding downtime

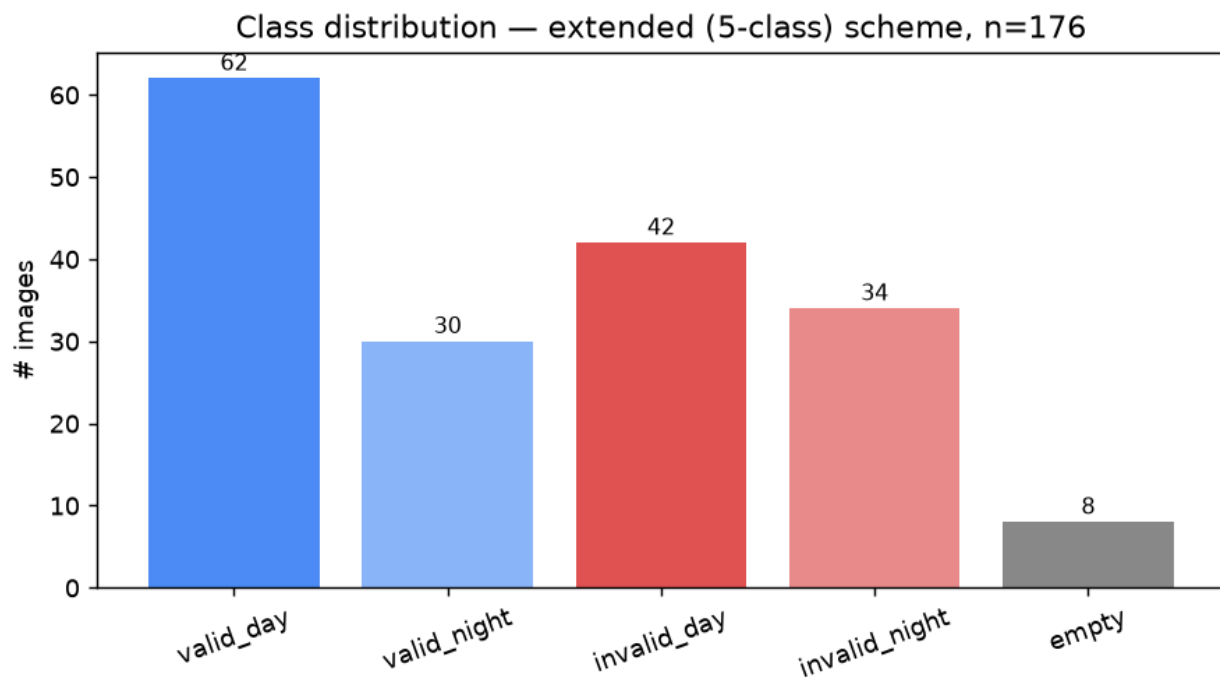
Yield-loss rate	\$2,000/hr per 1% P ₂ O ₅ → ~\$6,000/hr at a typical ~3% equivalent
Yield-loss exposure	~480 hr/yr affected (6% of ~8,000 hr/yr) → ~\$2,880,000/year
Downtime rate (equipment damage)	\$49,500/hr (45 × 1,100)
Downtime exposure	~240 hr/yr affected (3% of ~8,000 hr/yr) → ~\$11,880,000/year
Combined annual exposure	~\$14,760,000/year
Automated-detection recovery (10–15%)	~\$1,476,000 – \$2,214,000/year

Assumptions stated explicitly — swap in real annual operating hours for exact figures.

Related Work

- Transfer learning from ImageNet-pretrained CNNs is standard for small industrial inspection datasets — used in 83% of studies in a 2024 review of CNN-based surface-defect detection.
 - This project follows that pattern: MobileNetV2, transfer-learned, not trained from scratch.
- Class imbalance between “normal” and “defect” is treated as the norm in this literature — defects are genuinely rarer than normal output.
 - This project's “empty” class (8 of 176 images, 4.5%) sits in the same range; class-weighted loss is the standard mitigation, which this project uses.
- Published benchmarks reach 95%+ accuracy — but on datasets one to three orders of magnitude larger than this project's 176 images.
 - This project's ~77% CV accuracy should be read against that gap, not against best-case published numbers.

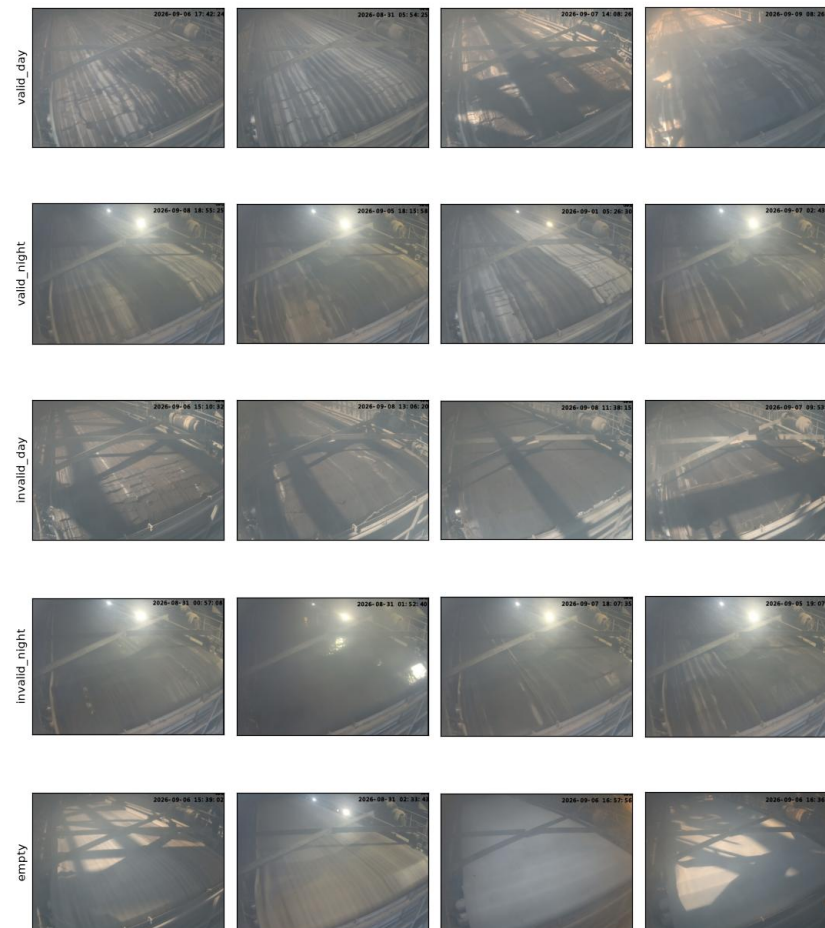
Dataset



- 176 photos total, from the plant's own camera (via WhatsApp export).
- 5 native classes: valid/invalid × day/night, plus empty filter.
- Randomized filenames + a manifest for traceability, not sequential/revealing names.
- Real capture-session timestamps recovered from the manifest — used for leakage-safe splitting (next slide).
- IT/data-access constraints limited the photos available for this phase — a full-scale pilot would use a much larger database (§16.2).

Dataset — What the Camera Actually Sees

Sample images per class (random, seed=42)



Preprocessing & Modeling Approach

- Timestamp overlay masked out (top strip) so the model can't key off the burned-in clock.
- Leakage-safe splits by real capture session (not per-image) — photos seconds apart don't leak across train/test.
- Three split ratios tested (80/20, 75/25, 70/30) plus K-fold cross-validation, auto-capped per scheme to what the scarcest class's session count actually supports.
- Model: MobileNetV2 (52 conv layers, ImageNet-pretrained) — transfer learning, not training from scratch (§1.1).
 - Only the last block + new classifier are unfrozen; the rest stays at pretrained weights — 176 images can't support fully fine-tuning 3.4M parameters without overfitting. A larger database would support unfreezing more (§16.3).
- Class-weighted CrossEntropyLoss for imbalance; a WeightedRandomSampler variant was also tried.
- One fixed config (backbone, lr=1e-3, 10 epochs) held constant across every comparison, so only one variable changes at a time.

Experiment 1: Which Label Scheme?

Cross-validation accuracy (the trustworthy number — averaged over independent sessions):

Scheme	CV Accuracy	Invalid Recall
Lite (3-class: valid/invalid/empty)	63.3% $\pm$ 9.5	42.4% $\pm$ 1.9
Extended (5-class: + day/night)	72.3% $\pm$ 0.5	63.9% $\pm$ 7.5
No-empty collapsed (2-class)	70.8% $\pm$ 14.3	57.7% $\pm$ 26.7 (unstable)
No-empty split (4-class)	77.4% $\pm$ 1.9	71.1% $\pm$ 5.4

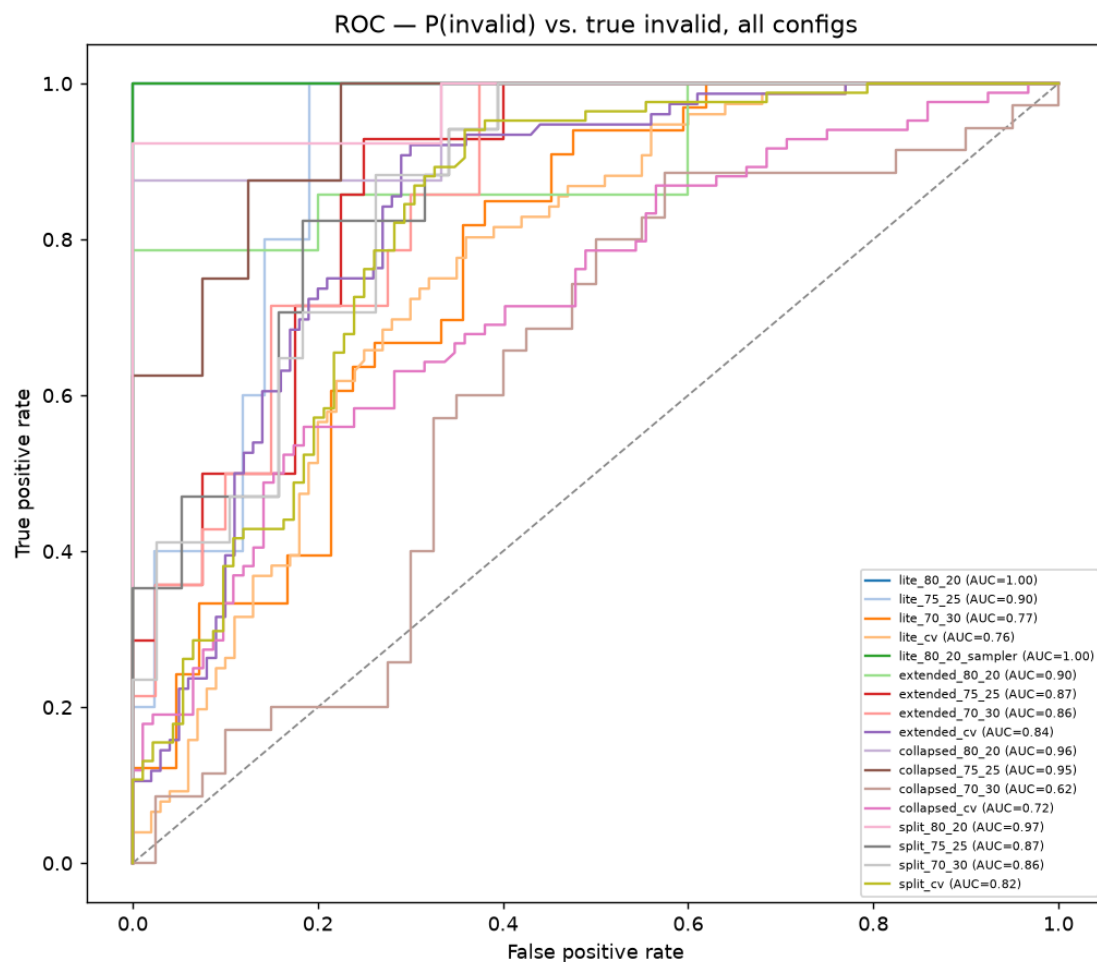
- Extended (5-class) beat Lite on every ratio and in CV — day/night carries real signal.
- No-empty split (4-class) beat every scheme tried — folding “empty” into “invalid” measurably helps, not just simplifies.
- No-empty collapsed's high single-split numbers are not trustworthy — its CV variance is enormous (± 14.3 points).

Decision: No-Empty Split (4-class)

- Compared fairly (same ground-truth definition, same run): no-empty split wins on every metric vs. extended — accuracy, invalid recall, ROC AUC (0.852 vs. 0.808), PR AP (0.805 vs. 0.769).
- Beyond the numbers — the operational argument is stronger:
 - The empty-filter dataset is tiny (8 of 176 images).
 - A live model doesn't need to detect “empty” at all — the operator empties the filter themselves and already knows it.
 - Production design: toggle the model off during that operator-controlled window, back on once filtration resumes.

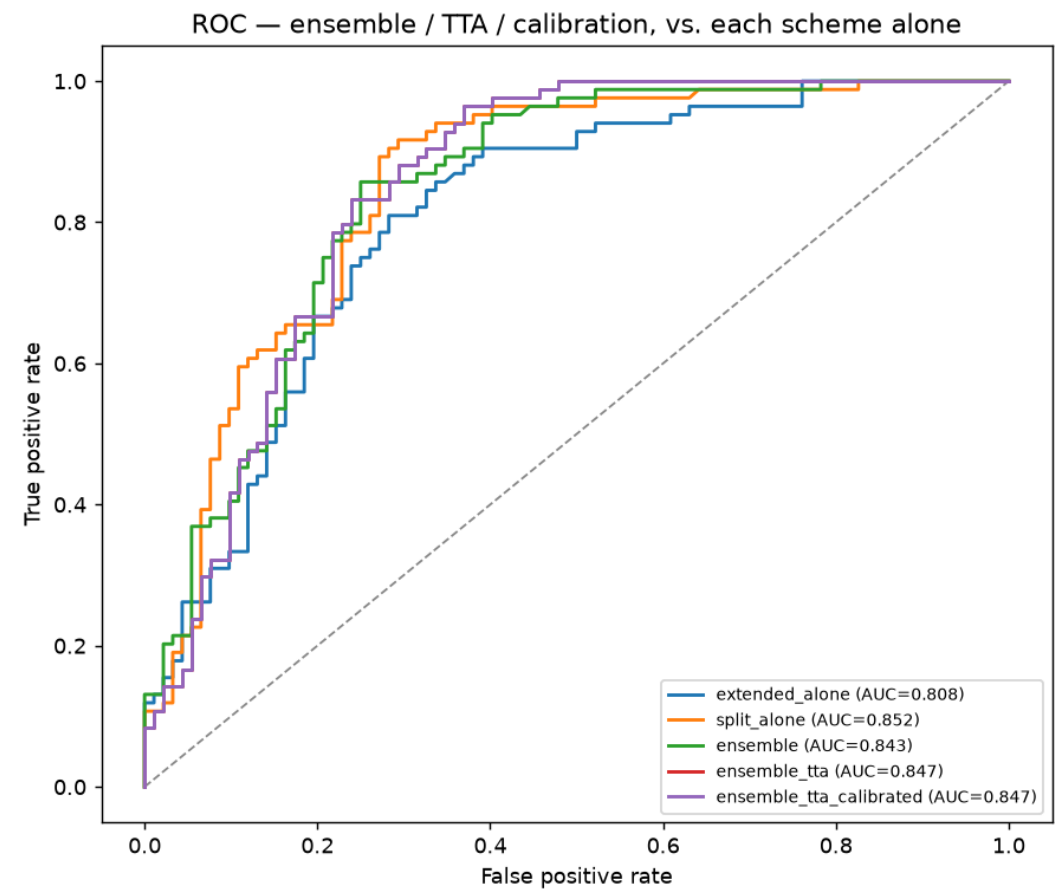
Classes: valid_day · valid_night · invalid_day · invalid_night

Experiment 2: Tuning the Decision Threshold



- Every training run so far decided valid-vs-invalid with a bare `argmax()` — an untuned, implicit 0.5 cutoff.
- Re-analyzed all 23 configs with full probabilities: ROC/AUC, precision-recall/AP, calibration, and two threshold candidates (Youden's J; F1-optimal).
- Finding: the default 0.5 cutoff was under-flagging real faults — nearly every config's optimal threshold is below 0.5.
- No-empty split CV: F1-optimal threshold = 0.466, close to 0.5.
- Extended CV: F1-optimal threshold = 0.302 — well below 0.5.

Experiment 3: Pushing Further (a Negative Result)



Variant	ROC AUC	PR AP
Extended alone	0.808	0.769
No-empty split alone	0.852	0.805
Ensemble (avg. of both)	0.843	0.803
+ Test-time augmentation	0.847	0.787
+ Calibration	0.847	0.787

- Tried: ensembling extended + no-empty split, test-time augmentation, Platt-scaling calibration.
- None beat the single best model — reported straight, not spun.
- A plain average pulled the stronger model down toward the weaker one.

Live Demo Scripts

- `predict.py` — classifies one photo from the command line: predicted class, confidence, and a VALID/INVALID call.
 - Auto-selects the best available trained model; “empty” predictions count as INVALID, matching the §12 decision.
 - Spot-checked on 3 sample photos: 2/3 correct at high confidence; the one miss landed on the model's documented weak spot (the empty class).
- `watch_camera_feed.py` — simulates the live feed: watches a folder, classifies each new photo within seconds, prints an ALERT line for INVALID.
 - A stand-in for a real camera connection — tested locally with a valid and an invalid sample photo, both classified and flagged correctly.
- Both reuse the exact same preprocessing as training — no drift between what was evaluated and what runs live.

Conclusions

- No-empty split (4-class) is the strongest model found — ~77% CV accuracy, ROC AUC 0.852, clearly ahead once compared fairly.
- Folding “empty” into “invalid” is both statistically and operationally correct, not just a simplification.
- The default 0.5 threshold was under-flagging real faults — tuned thresholds catch more real invalid cases.
- Ensembling/TTA/calibration did not beat the single best model — an honest negative result.
- Business case: automated detection is estimated to recover ~\$1.48M–2.21M/year (10–15% of ~\$14.76M/year combined yield-loss + downtime exposure).

Limitations

- Small dataset by IT constraint, not choice — 176 images (8 “empty”); every number carries real uncertainty.
- Single seed, mostly CPU-trained runs.
- The promoted model hasn't had a real full training run yet — `full_pipeline.ipynb` still needs to run in Colab for real.
- Day/night remains unexplained — ruled out literal time-of-day and whole-image brightness, but the real cause is still unknown.
- The live-feed demo is a simulation (folder-watcher), not a real camera/RTSP integration or real alerting channel.
- Single fixed camera, uncontrolled lighting; source images are WhatsApp-recompressed, not native resolution.

Further Directions

Near-term

- Run full_pipeline.ipynb for real to get trustworthy final numbers and the actual promoted weights.
- Confidence-weighted ensembling instead of a plain average; build the real camera feed + alerting channel.

Full-scale pilot (plant engineer's roadmap)

- Retrain on a much larger database once IT/data-access constraints are lifted, and unfreeze more of the backbone (today's last-block-only fine-tuning is a dataset-size-driven choice, not a ceiling).
- Add projectors around the filter to normalize day/night lighting and reduce shadow effects — engineers the day/night ambiguity away at the source.
- Move beyond binary valid/invalid to a continuous filtration-quality score, so the process trend is visible.
- Make it a time-based model that tracks change over a sequence of frames and flags the process turning bad — early warning, not single-frame classification.
- Far future: a downstream model that calculates the right reactor process parameters from the predicted filtration state — closing the loop from vision to process control.

Thank you

Questions?

Code: github.com/yaronconroy-del/picture-recognition-gypsum